

Vectors and Embeddings - Beginner Guide

1. What is a Vector?

A vector is simply a list of numbers arranged in a specific order.

Example:

[2, 5, 7]

or

[0.12, -0.44, 0.98]

In machine learning and artificial intelligence, vectors are used to represent data numerically.

You can think of a vector as a point in space.

Example:

[3,5] means a point that is 3 units on the X-axis and 5 units on the Y-axis.

2. Visualizing a Vector

Imagine a simple graph with X and Y axis.

Vector [3,5] represents the point (3,5).

Example:

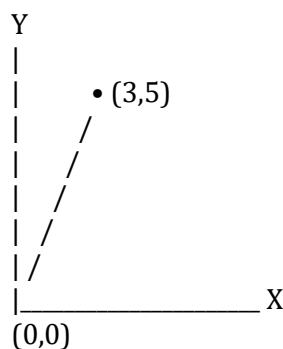
Move 3 steps to the right and 5 steps up.

In AI systems vectors often contain hundreds or thousands of numbers.

For example:

[0.21, 0.55, 0.98, 0.11, ...]

These represent positions in high dimensional space.



3. Why Vectors Are Important

Computers do not understand human language directly. They understand numbers. Therefore words, sentences and images are converted into vectors.

Example:

Dog -> [0.23, 0.91, 0.44, 0.11]

Cat -> [0.21, 0.89, 0.40, 0.09]

Car -> [0.77, 0.10, 0.21, 0.55]

Notice that Dog and Cat vectors are similar while Car is very different.

This means vectors can represent meaning.

4. What are Embeddings?

An embedding is a vector representation of something meaningful.

It converts text, images, or objects into numerical form so that computers can compare them.

Example:

Text:

"Apple is a fruit"

Embedding:

[0.13, 0.91, 0.45, 0.67, 0.22 ...]

This vector captures the meaning of the sentence.

5. Key Idea of Embeddings

The most important concept of embeddings is:

Similar things should have similar vectors.

Example:

cat -> [0.8, 0.3, 0.1]

dog -> [0.7, 0.4, 0.1]

car -> [0.1, 0.2, 0.9]

truck -> [0.2, 0.1, 0.8]

Here:

cat and dog are close in vector space.

car and truck are close in vector space.

6. Famous Embedding Example

Embeddings can capture relationships.

Example:

king - man + woman $\approx$ queen

This means the system understands relationships between concepts.

7. Types of Embeddings

Text Embeddings

Convert text into vectors.

Example: "I love dogs"

Image Embeddings

Convert images into vectors.

Example: photo of a cat.

User / Product Embeddings

Used in recommendation systems.

Example: recommending movies similar to user interests.

8. Measuring Similarity

To measure similarity between vectors we use Cosine Similarity.

Formula:

Cosine Similarity = (A dot B) / (|A| * |B|)

Value Range:

1 = very similar

0 = unrelated

-1 = opposite meaning

Example:

cat = [0.8, 0.3] → Represents vector representation of cat

dog = [0.7, 0.4] → Represents vector representation of dog

Here, Cosine similarity ≈ 0.98

Calculation:

$(0.8 \times 0.7) + (0.3 \times 0.4)$

$0.56 + 0.12$

Dot Product = 0.68

Magnitude means **vector length**.

Formula:

$$|A| = \sqrt{x^2 + y^2}$$

Magnitude of cat

$\sqrt{(0.8^2 + 0.3^2)}$

$\sqrt{(0.64 + 0.09)}$

$\sqrt{0.73} \approx 0.854$

Magnitude of dog

$$\sqrt{(0.7^2 + 0.4^2)}$$

$$\sqrt{(0.49 + 0.16)}$$

$$\sqrt{0.65} \approx 0.806$$

$$\text{Cosine Similarity} = 0.68/0.854 * 0.806 = \sim 0.988$$

This means cat and dog are very similar.

9. Embeddings in AI Systems

Embeddings are used in many modern AI applications.

Semantic Search

Semantic Search is a search technique where the system tries to understand the **meaning of the query**, not just the exact words.

Instead of matching keywords, the system finds similar meaning.

Semantic search uses **embeddings**.

Instead of comparing words, it compares **vector meaning**.

Workflow:

User Query



Convert query to embedding



Search similar embeddings in database



Return documents with similar meaning

Example of Embeddings

Example 1 :- Embeddings in Search Engines

Suppose a user searches on Google: "*How can I repair my car engine?*"

Now imagine a document on a website contains the title: "*Tips for fixing vehicle engines*"

If a search engine used **only keyword matching**, it might struggle because:

Query Word	Document Word
Repair	Fixing
car	Vehicle
engine	Engine

Two words are different:
repair $\neq$ fixing
car $\neq$ vehicle
But humans know these mean almost the same thing.

How Embeddings Solve This Problem

Modern search engines convert text into embeddings (vectors).

Step 1 — Convert the query into an embedding

User query:

"How can I repair my car engine?"

Embedding vector (example):

[0.82, 0.14, 0.67, 0.23, 0.55]

Step 2 — Convert documents into embeddings

Document:

"Tips for fixing vehicle engines"

Embedding vector:

[0.80, 0.18, 0.65, 0.21, 0.53]

Step 3 — Compare vectors

The vectors are very close to each other.

So the system calculates **cosine similarity** and gets a high score.

Example:

Cosine Similarity $\approx$ 0.95

This tells the search engine:

These sentences have very similar meaning.

So the document is returned in search results.

Example 2 :- Embeddings in Recommendation Systems (Netflix and Spotify)

Netflix and Spotify recommend items using embeddings.

A Recommendation System suggests items to users based on their preferences.

Examples:

- Netflix recommending movies
- Spotify recommending songs
- Amazon recommending products
- YouTube recommending videos

These systems try to answer a simple question:

"What will the user like next?"

To do this, modern systems use embeddings (vectors).

How Embeddings Help Recommendations

Embeddings convert things into vectors of numbers.

Examples of things that can be converted into embeddings:

- movies
- songs
- products
- users

Example movie embeddings:

Action Movie → [0.92, 0.10, 0.35]

Romantic Movie → [0.11, 0.88, 0.30]

Sci-Fi Movie → [0.87, 0.20, 0.50]

Comedy Movie → [0.40, 0.60, 0.70]

These numbers represent **characteristics of the item**, such as:

- genre
- mood
- style
- popularity
- audience preference

Items with **similar vectors are considered similar**.

How Netflix Recommendation actually works?

Suppose a user watches these movies on Netflix:

- Interstellar
- The Matrix
- Inception

These movies are represented using **embeddings (vectors)**.

Example embeddings:

Interstellar → [0.90, 0.15, 0.40]

The Matrix → [0.85, 0.20, 0.50]

Inception → [0.88, 0.18, 0.45]

Each number represents hidden characteristics of the movie, such as:

- genre
- story complexity
- action level
- sci-fi themes
- emotional tone

These are learned automatically by machine learning models.

What These Numbers Represent

Think of the vector like coordinates describing the movie.

Example interpretation:

Feature	Meaning
1st number	Sci-fi level
2nd number	action intensity
3rd number	story complexity

Example:

Interstellar → [0.90, 0.15, 0.40]

Meaning:

- very strong sci-fi
- moderate action
- complex story

Creating the User Preference Vector

The system combines the embeddings of movies the user watched.

The simplest method is **averaging the vectors**.

Step 1 — Add vectors

Interstellar → [0.90, 0.15, 0.40]

The Matrix → [0.85, 0.20, 0.50]

Inception → [0.88, 0.18, 0.45]

Add them:

[0.90 + 0.85 + 0.88,

0.15 + 0.20 + 0.18,

0.40 + 0.50 + 0.45]

Result:

[2.63, 0.53, 1.35]

Step 2 — Divide by number of movies

There are 3 movies.

User Vector =

[2.63/3, 0.53/3, 1.35/3]

Result:

User Vector → **[0.88, 0.18, 0.45]**

This vector represents the **user's overall taste**.

What the User Vector Means

User Vector → [0.88, 0.18, 0.45]

This tells the system the user likes movies that are:

- high in **sci-fi**
- moderate **action**
- medium **story complexity**

This vector becomes the **user's preference profile**.

Searching the Movie Database

After the system builds the user preference vector, it must find movies that match the user's taste.

From your example:

User Vector → [0.88, 0.18, 0.45]

This vector represents the type of movies the user likes.

Now imagine a database (like the one used by Netflix) containing embeddings for thousands of movies.

Example movie embeddings:

Movie	Vector
○ Sci-Fi Movie	[0.89, 0.17, 0.46]
○ Romantic Movie	[0.20, 0.85, 0.30]
○ Action Sci-Fi	[0.87, 0.19, 0.44]

Each vector describes characteristics of that movie.

For example (simplified interpretation):

Feature	Meaning
○ First number	Sci-Fi level
○ Second number	Romance / emotional level
○ Third number	Action / story complexity

So:

Sci-Fi Movie → [0.89, 0.17, 0.46]

means:

- very strong sci-fi
- low romance
- medium complexity

What the system does :

The recommendation system compares User Vector with every movie vector in the database

To measure how similar they are, the system calculates cosine similarity.

Cosine similarity tells us:

How close two vectors are in direction

If vectors point in almost the same direction, similarity is close to 1.

Example Recommendation Result

Now let's compare the user vector with three movies.

User vector

[0.88, 0.18, 0.45]

Movie embeddings

Movie A → [0.89, 0.17, 0.46]

Movie B → [0.20, 0.85, 0.30]

Movie C → [0.87, 0.19, 0.44]

Understanding Similarity Scores

The system calculates cosine similarity between the user vector and each movie vector.

Result:

	Movie	Similarity
○	Movie A	0.99
○	Movie C	0.98
○	Movie B	0.32

Let's understand what this means.

Movie A

Movie A $\rightarrow$ [0.89, 0.17, 0.46]

User $\rightarrow$ [0.88, 0.18, 0.45]

These numbers are almost identical.

Difference is very small:

$0.89 \approx 0.88$

$0.17 \approx 0.18$

$0.46 \approx 0.45$

So cosine similarity becomes: 0.99 (almost perfect match). Meaning: $\rightarrow$ This movie is very similar to what the user likes.

Movie C

Movie C $\rightarrow$ [0.87, 0.19, 0.44]

User $\rightarrow$ [0.88, 0.18, 0.45]

These are also very close.

Similarity: 0.98

Meaning: $\rightarrow$ This movie is also very similar to the user's taste.

Movie B

Movie B $\rightarrow$ [0.20, 0.85, 0.30]

User $\rightarrow$ [0.88, 0.18, 0.45]

Now look at the numbers.

They are very different.

0.20 vs 0.88

0.85 vs 0.18

0.30 vs 0.45

This means the movie represents a completely different style.

Similarity becomes: 0.32 (low similarity)

Meaning: $\rightarrow$ This movie does not match the user's preference.

Movies Recommended to the User

The system recommends:

Movie A

Movie C

Because their vectors are closest to the user vector.

These movies are expected to match the user's taste.

So in recommendation systems, each movie is converted into an embedding vector that represents its characteristics. When a user watches multiple movies, the system combines their embeddings to create a user preference vector. The platform then searches for movies whose embeddings are closest to the user vector and recommends them

Conclusion: Vectors and Embeddings

Vectors and embeddings are the foundation of modern AI systems. Since computers cannot understand human language directly, text, images, and other data must be converted into numerical representations.

In summary, vectors provide the mathematical structure, while embeddings capture the semantic meaning of data. Together, they allow machines to understand relationships between words, documents, and concepts, forming the basis of modern AI-powered search, recommendation, and conversational systems.